Plain C AES-128 Documentation

Serial Implementation Report

Made by Rayyan Hassan Salman
2023601, Computer Science Junior

Faculty of Computer Sciences and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Project Context: This document describes the modular plain C implementation of AES-128 used as the serial baseline for a CUDA-accelerated AES project. The implementation separates concerns into IO handling, AES core utilities, encryption flow, and decryption flow.

Repository Link to This Work

https://github.com/Rayy1704/cuda_accelerated_AES/tree/main/c-implementation

April 2, 2026

Contents

1 Input and Output Module **2**
- 1.1 Key Responsibilities . . . 2
- 1.2 Design Notes . . . 2
- 1.3 Snippet: Hex Text to Byte Buffer Parsing . . . 2
- 1.4 Snippet: Key File Parsing Outline . . . 3

2 Core Functions **4**
- 2.1 Architecture and Interfaces . . . 4
- 2.2 Core Responsibilities . . . 4
- 2.3 Snippet: State Population in Column-Major Order . . . 4
- 2.4 Snippet: AddRoundKey Transformation . . . 5
- 2.5 Snippet: GF($2\hat{8}$) Multiplication Helper . . . 5

3 Key Expansion **5**
- 3.1 Role in the Pipeline . . . 5
- 3.2 Snippet: Key Expansion Core Logic . . . 6

4 Encryption **6**
- 4.1 Encryption Stages . . . 6
- 4.2 Round Structure . . . 6
- 4.3 Snippet: Column Mix Operation . . . 7
- 4.4 Snippet: Full Encryption Round Flow . . . 7

5 Decryption **8**
- 5.1 Decryption Stages . . . 8
- 5.2 Round Structure . . . 8
- 5.3 Snippet: Inverse ShiftRows . . . 8
- 5.4 Snippet: Full Decryption Round Flow . . . 9

6 Main Program and Makefile **9**
- 6.1 Main Program Flow . . . 10
- 6.2 Snippet: Complete Main Control Flow (Abbreviated Error Blocks) . . . 10
- 6.3 Makefile Responsibilities . . . 11

Summary **11**

1 Input and Output Module

The IO layer was designed to stay independent from cryptographic transformations. It is linked through `io.h`, while AES-related modules are connected through `aes.h`. This separation keeps file parsing logic out of encryption/decryption code.

For AES-128 specifically, input correctness is essential because the algorithm works at byte granularity on a fixed 16-byte state. If hexadecimal parsing or key parsing is incorrect by even one nibble, every subsequent round transformation produces invalid ciphertext/plaintext. Therefore, this module is not just utility code; it is foundational to cryptographic correctness.

1.1 Key Responsibilities

- Read plaintext/ciphertext from text files containing hexadecimal bytes.
- Read the AES key from a separate key file.
- Convert hex-string text input into raw byte values.
- Write computed output bytes back to a text file as hexadecimal.

1.2 Design Notes

- The data reader determines approximate binary length using file size divided by 3, because each byte is represented as two hex digits plus a separator.
- Parsing uses a nibble-combine strategy: the high nibble is left-shifted by 4 and OR-ed with the low nibble.
- The key reader is separated from generic data reading and is designed to parse exactly a 16-byte AES-128 key from text.
- Output writing emits bytes in hexadecimal format for easy inspection.

The nibble merge operation maps directly to AES byte semantics. AES transformations such as SubBytes, ShiftRows, and MixColumns operate on bytes over $GF(2^8)$, so preserving exact byte values from text input is mandatory. In the same way, writing output back as hex keeps the binary result human-readable while preserving exact values for test-vector comparison.

1.3 Snippet: Hex Text to Byte Buffer Parsing

This snippet is relevant to AES-128 because it reconstructs each cryptographic byte from text form. The expression (`high_nibble` << 4)|`val` restores the original 8-bit value that AES rounds consume.

```
size_t byte_count = 0;
int high_nibble = -1;

for (long i = 0; i < size; i++) {
    int val = hex_to_int(temp_text[i]);
    if (val != -1) {
        if (high_nibble == -1) {
            high_nibble = val;
        } else {
```

```
            buf[byte_count++] = (high_nibble << 4) | val;
            high_nibble = -1;
        }
    }
}
```

Listing 1: Core parsing idea used in file readers

1.4 Snippet: Key File Parsing Outline

This flow matters because AES-128 requires an exact 128-bit key (16 bytes). A dedicated parser ensures the key path is deterministic and isolated from generic input parsing.

```c
unsigned char * read_key_file(const char *keyFileName) {
    FILE *fd = fopen(keyFileName, "r");
    //error handiling for key file open failure

    unsigned char *temp_buf = malloc(49);
    //error handiling for temporary key buffer allocation

    size_t bytes_read = fread(temp_buf, 1, 48, fd);
    //error handiling for short/invalid key input read

    temp_buf[48] = '\0';
    fclose(fd);

    unsigned char *initial_key = malloc(16);
    //error handiling for final key allocation

    // Parse two hex nibbles into one byte
    size_t byte_count = 0;
    int high_nibble = -1;
    for (long i = 0; i < 48; i++) {
        int val = hex_to_int(temp_buf[i]);
        if (val != -1) {
            if (high_nibble == -1) high_nibble = val;
            else {
                initial_key[byte_count++] = (high_nibble << 4) | val;
                high_nibble = -1;
            }
        }
    }

    free(temp_buf);
    return initial_key;
}
```

Listing 2: Dedicated 16-byte key reader flow

2 Core Functions

The AES core module hosts shared logic used by both encryption and decryption, and also functions required by `main`. This includes state population, key schedule generation, finite field arithmetic helpers, round key application, and S-box data.

In AES-128, the core layer reflects the mathematical skeleton of the cipher. The block size is fixed at 128 bits (16 bytes), the key size is 128 bits, and the algorithm runs 10 rounds after key expansion. The core functions implement these invariants in reusable form so both encrypt and decrypt modules remain consistent.

2.1 Architecture and Interfaces

- `aes.h` provides common declarations used across encryption, decryption, and main.
- `sbox` is defined in the core file and declared `extern` in the header so other modules can use it.
- The state is represented as a 4x4 byte matrix in column-major form.

The 4x4 matrix representation is directly aligned with the AES specification. Bytes are arranged by columns, which is why state loading and state write-back explicitly use column-major indexing. This design minimizes conceptual mismatch between code and algorithm.

2.2 Core Responsibilities

- **populate_state**: maps consecutive 16-byte input into AES state matrix columns.
- **rot_word**: rotates a 4-byte word left by one byte.
- **sub_word**: substitutes each byte in a word through S-box.
- **rcon**: XORs first word byte with round constant.
- **xtime**: multiplication by 2 in $GF(2^8)$.
- **multiply**: helper for MixColumns and InvMixColumns multiplications.
- **add_round_key**: XORs selected round key into state.

2.3 Snippet: State Population in Column-Major Order

State population is relevant because every AES round function assumes this exact arrangement. If row-major order were used accidentally, ShiftRows and MixColumns would operate on the wrong byte positions.

```
for (int c = 0; c < 4; c++) {
    for (int r = 0; r < 4; r++) {
        state->matrix[r][c] = input[c * 4 + r];
    }
}
```

Listing 3: Mapping linear bytes into AES state matrix

2.4 Snippet: AddRoundKey Transformation

AddRoundKey is the key-dependent non-linear driver of AES security from a system perspective. Each round key changes diffusion trajectories, making this XOR step cryptographically central rather than a simple utility operation.

```c
void add_round_key(aes_state *state, unsigned char *expanded_keys, int round) {
    unsigned char *round_key = &expanded_keys[round * 16];
    for (int r = 0; r < 4; r++) {
        for (int c = 0; c < 4; c++) {
            state->matrix[r][c] ^= round_key[r + c * 4];
        }
    }
}
```

Listing 4: Round-key XOR into state matrix

2.5 Snippet: GF(2$\hat{8}$) Multiplication Helper

MixColumns and InvMixColumns rely on finite-field multiplication, not integer multiplication. This helper encodes AES-specific multiplication constants used by the mix matrices.

```c
unsigned char multiply(unsigned char a, unsigned char b) {
    if (a == 1) return b;
    if (a == 2) return xtime(b);
    if (a == 3) return xtime(b) ^ b;
    if (a == 0x09) return xtime(xtime(xtime(b))) ^ b;
    if (a == 0x0b) return xtime(xtime(xtime(b))) ^ xtime(b) ^ b;
    if (a == 0x0d) return xtime(xtime(xtime(b))) ^ xtime(xtime(b)) ^ b;
    if (a == 0x0e) return xtime(xtime(xtime(b) ^ b)) ^ xtime(b);
    return 0;
}
```

Listing 5: Finite-field multiply cases used by MixColumns

3 Key Expansion

Key expansion is handled in a dedicated routine before encryption or decryption begins. Starting from the initial 16-byte AES-128 key, the implementation computes the complete round-key schedule required by both directions of transformation.

For AES-128, key expansion generates 11 round keys of 16 bytes each (one for initial whitening and ten for rounds). This derived schedule prevents direct reuse of raw key material every round and ensures each round contributes unique key-dependent behavior.

3.1 Role in the Pipeline

- Runs once after key file parsing and before block processing.
- Allocates memory for all 11 round keys (176 bytes total).
- Uses `rot_word`, `sub_word`, and `rcon` every 16 bytes.

- Returns the expanded key buffer to `main`, which later frees it.

3.2 Snippet: Key Expansion Core Logic

This snippet captures the AES-128 schedule recurrence. Every 16-byte boundary applies the key schedule core (RotWord, SubWord, Rcon), then XOR chaining derives the next words.

```
for (int i = 16; i < 176; i += 4) {
    unsigned char temp[4];

    for (int j = 0; j < 4; j++) {
        temp[j] = expanded_keys[i - 4 + j];
    }

    if (i % 16 == 0) {
        rot_word(temp);
        sub_word(temp);
        rcon(temp, i / 16);
    }

    for (int j = 0; j < 4; j++) {
        expanded_keys[i + j] = expanded_keys[i - 16 + j] ^ temp[j];
    }
}
```

Listing 6: AES-128 key schedule generation loop

4 Encryption

The encryption implementation is isolated in its own module with constants declared globally. Each AES transformation stage is implemented as an individual function to keep logic clear and debuggable.

AES-128 encryption transforms each 16-byte plaintext block into ciphertext through substitution, permutation, and diffusion operations. The round structure is designed so local byte changes propagate across the full state after repeated rounds, improving confusion and diffusion.

4.1 Encryption Stages

- **SubBytes**: substitutes each state byte via S-box.
- **ShiftRows**: left shifts row 1, 2, and 3 by 1, 2, and 3 positions.
- **MixColumns**: transforms each column using Galois-field matrix multiplication.
- **AddRoundKey**: XORs the round key into the state.

4.2 Round Structure

- Initial round applies AddRoundKey.
- Nine main rounds apply SubBytes, ShiftRows, MixColumns, AddRoundKey.
- Final round applies SubBytes, ShiftRows, AddRoundKey (without MixColumns).

- Final encrypted state is written back into the original data buffer.

The omission of MixColumns in the final round is part of standard AES design and ensures that encryption and decryption remain structurally invertible while preserving expected test vectors.

4.3 Snippet: Column Mix Operation

This operation applies a fixed matrix over $GF(2^8)$ to each column, enabling inter-byte diffusion inside columns. It is one of the most mathematically AES-specific steps in the implementation.

```
for (int i = 0; i < 4; i++) {
    unsigned char col[4];
    for (int j = 0; j < 4; j++) {
        col[j] = state->matrix[j][i];
    }

    for (int j = 0; j < 4; j++) {
        state->matrix[j][i] =
             multiply(mix_matrix[j][0], col[0])
            ^multiply(mix_matrix[j][1], col[1])
            ^multiply(mix_matrix[j][2], col[2])
            ^multiply(mix_matrix[j][3], col[3]);
    }
}
```

Listing 7: MixColumns operation per state column

4.4 Snippet: Full Encryption Round Flow

This full flow demonstrates how AES-128 rounds are sequenced in code. It links all per-stage functions into the exact operational pipeline for each block.

```
for (size_t t = 0; t < len; t += 16) {
    populate_state(&state, data + t);

    add_round_key(&state, expanded_keys, 0);

    for (int i = 0; i < 9; i++) {
        sub_bytes(&state);
        shift_rows(&state);
        mixcolumns(&state);
        add_round_key(&state, expanded_keys, i + 1);
    }

    sub_bytes(&state);
    shift_rows(&state);
    add_round_key(&state, expanded_keys, 10);

    for (int c = 0; c < 4; c++) {
        for (int r = 0; r < 4; r++) {
            data[t + c * 4 + r] = state.matrix[r][c];
        }
```

```
    }
}
```

Listing 8: Block-level AES-128 encryption flow

5 Decryption

The decryption module mirrors encryption with inverse transformations and inverse constants declared globally. Similar to encryption, each stage is separated into dedicated functions.

AES-128 decryption applies the inverse of each transformation in reverse order. Correctness depends on exact inversion of byte substitution, row permutation, and column mixing under the same expanded key schedule.

5.1 Decryption Stages

- **InvSubBytes**: substitutes each byte through inverse S-box.
- **InvShiftRows**: right shifts row 1, 2, and 3 by 1, 2, and 3 positions.
- **InvMixColumns**: multiplies each column with inverse mix matrix in $GF(2^8)$.
- **AddRoundKey**: applies appropriate round key by XOR.

5.2 Round Structure

- Starts with AddRoundKey using the final round key.
- Runs inverse rounds from round 9 down to 1.
- Ends with inverse row shift, inverse substitution, and round-0 AddRoundKey (without InvMixColumns).
- Writes the recovered plaintext back to the same data buffer.

Using round keys in reverse order is essential to recover plaintext exactly. This is why the same key expansion output can be reused for both encryption and decryption without recomputing inverse keys separately.

5.3 Snippet: Inverse ShiftRows

Inverse row shifts restore byte alignment disturbed during encryption ShiftRows, re-aligning state bytes before inverse substitution and key mixing.

```c
// Row 1 right shift by 1
temp = state->matrix[1][3];
state->matrix[1][3] = state->matrix[1][2];
state->matrix[1][2] = state->matrix[1][1];
state->matrix[1][1] = state->matrix[1][0];
state->matrix[1][0] = temp;

// Row 2 right shift by 2
unsigned char t0 = state->matrix[2][2];
unsigned char t1 = state->matrix[2][3];
```

```
state->matrix[2][3] = state->matrix[2][1];
state->matrix[2][2] = state->matrix[2][0];
state->matrix[2][0] = t0;
state->matrix[2][1] = t1;

// Row 3 right shift by 3
temp = state->matrix[3][0];
state->matrix[3][0] = state->matrix[3][1];
state->matrix[3][1] = state->matrix[3][2];
state->matrix[3][2] = state->matrix[3][3];
state->matrix[3][3] = temp;
```

Listing 9: Row-wise inverse cyclic shifts

5.4 Snippet: Full Decryption Round Flow

This block-level flow shows decryption symmetry with encryption while preserving the strict inverse ordering required by AES-128.

```
for (size_t i = 0; i < len; i += 16) {
    populate_state(&state, data + i);

    add_round_key(&state, expanded_keys, 10);

    for (int round = 9; round > 0; round--) {
        inv_shift_rows(&state);
        inv_sub_bytes(&state);
        add_round_key(&state, expanded_keys, round);
        inv_mix_columns(&state);
    }

    inv_shift_rows(&state);
    inv_sub_bytes(&state);
    add_round_key(&state, expanded_keys, 0);

    for (int r = 0; r < 4; r++) {
        for (int c = 0; c < 4; c++) {
            data[i + c * 4 + r] = state.matrix[r][c];
        }
    }
}
```

Listing 10: Block-level AES-128 decryption flow

6 Main Program and Makefile

The serial application is orchestrated through `main` and compiled via a focused Makefile setup.

This layer is relevant to AES because it binds algorithmic primitives into a usable cryptographic toolchain: argument parsing, key/data loading, operation selection, and final output persistence.

6.1 Main Program Flow

- Validates command-line argument count and prints usage on invalid input.
- Extracts input file, key file, and optional output filename.
- Reads key and data buffers with error checking and proper cleanup on failure.
- Expands the key schedule once before AES execution.
- Prompts user to choose encryption or decryption.
- Writes final processed buffer to output file.
- Frees allocated buffers and exits cleanly.

6.2 Snippet: Complete Main Control Flow (Abbreviated Error Blocks)

The snippet below reflects complete execution flow while intentionally abbreviating verbose error branches into purpose comments, keeping architectural clarity without clutter.

```c
int main(int argc, char *argv[]) {
    if (argc < 3 || argc > 4) {
        //error handiling for invalid argument count and usage message
        return 1;
    }

    const char *fileName = argv[1];
    const char *keyFileName = argv[2];
    const char *out_filename = (argc == 4) ? argv[3] : "output.txt";
    size_t data_len = 0;

    unsigned char *key_buf = read_key_file(keyFileName);
    if (key_buf == NULL) {
        //error handiling for key file read failure
        return 1;
    }

    unsigned char *buf = read_file(fileName, &data_len);
    if (buf == NULL) {
        //error handiling for data file read failure and key cleanup
        return 1;
    }

    unsigned char *expanded_keys_ptr = key_expansion(key_buf);
    if (expanded_keys_ptr == NULL) {
        //error handiling for key expansion failure and buffer cleanup
        return 1;
    }

    printf("1) Encrypt\\n2) Decrypt\\nChoose an option: ");
    int choice;
    scanf("%d", &choice);

    if (choice == 1) {
```

```
        aes_encrypt(buf, expanded_keys_ptr, data_len);
    } else if (choice == 2) {
        aes_decrypt(buf, expanded_keys_ptr, data_len);
    }

    write_buf_to_file(buf, data_len, out_filename);

    //error handiling for invalid menu choice can be inserted here
    //error handiling for output write failure can be inserted here

    free(expanded_keys_ptr);
    free(buf);
    free(key_buf);
    return 0;
}
```

Listing 11: Main pipeline with purpose comments for error handling

6.3 Makefile Responsibilities

- Compiles all source modules with include paths and warning flags.
- Provides an `all` target to build the executable.
- Provides a `run` target to execute with project input/output paths.
- Provides a `clean` target to remove generated artifacts.

From an AES workflow perspective, the Makefile guarantees reproducible builds and repeatable execution paths. This is important when validating cryptographic behavior across test inputs and when comparing CPU baseline results against CUDA-accelerated versions.

Summary

This serial AES-128 implementation follows a clear modular approach:

- IO is isolated from cryptographic logic.
- AES core holds reusable primitives and key schedule machinery.
- Encryption and decryption are each staged and independently testable.
- Main and Makefile provide complete end-to-end execution and build control.

This organization makes the code easier to maintain, debug, and use as a reliable CPU baseline before moving to CUDA acceleration.